

22. ROS2 Gazebo Simulation Platform

Note: The accompanying reference video uses the ROS Humble version, for reference only

1. Gazebo Overview

Gazebo is the most commonly used **3D physics simulation platform** in the ROS system. It supports dynamics engines and can achieve high-quality graphics rendering. It can not only simulate robots and their surrounding environment but also add physical properties such as friction coefficients and elasticity.

For example, if we want to develop a Mars rover, we can simulate the Mars surface environment in Gazebo. Another example is drone development. Due to limited battery life and flight restrictions, we can't frequently conduct experiments with physical drones. At this time, we might as well use Gazebo for simulation first. Once the algorithm development is mostly complete, deploy it to the physical device for operation.

Therefore, simulation platforms like Gazebo can help us verify robot algorithms, optimize robot design, and test robot application scenarios, providing more possibilities for robot development.

Note: This section is only for learning. The environment is not configured in the tutorial because we directly use physical device debugging here.

2. Installation and Running

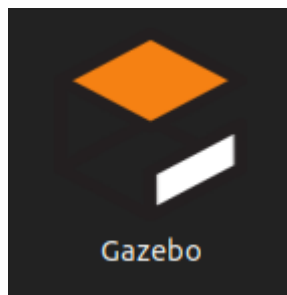
- Install gazebo via apt command:

```
sudo apt install gz-harmonic
```

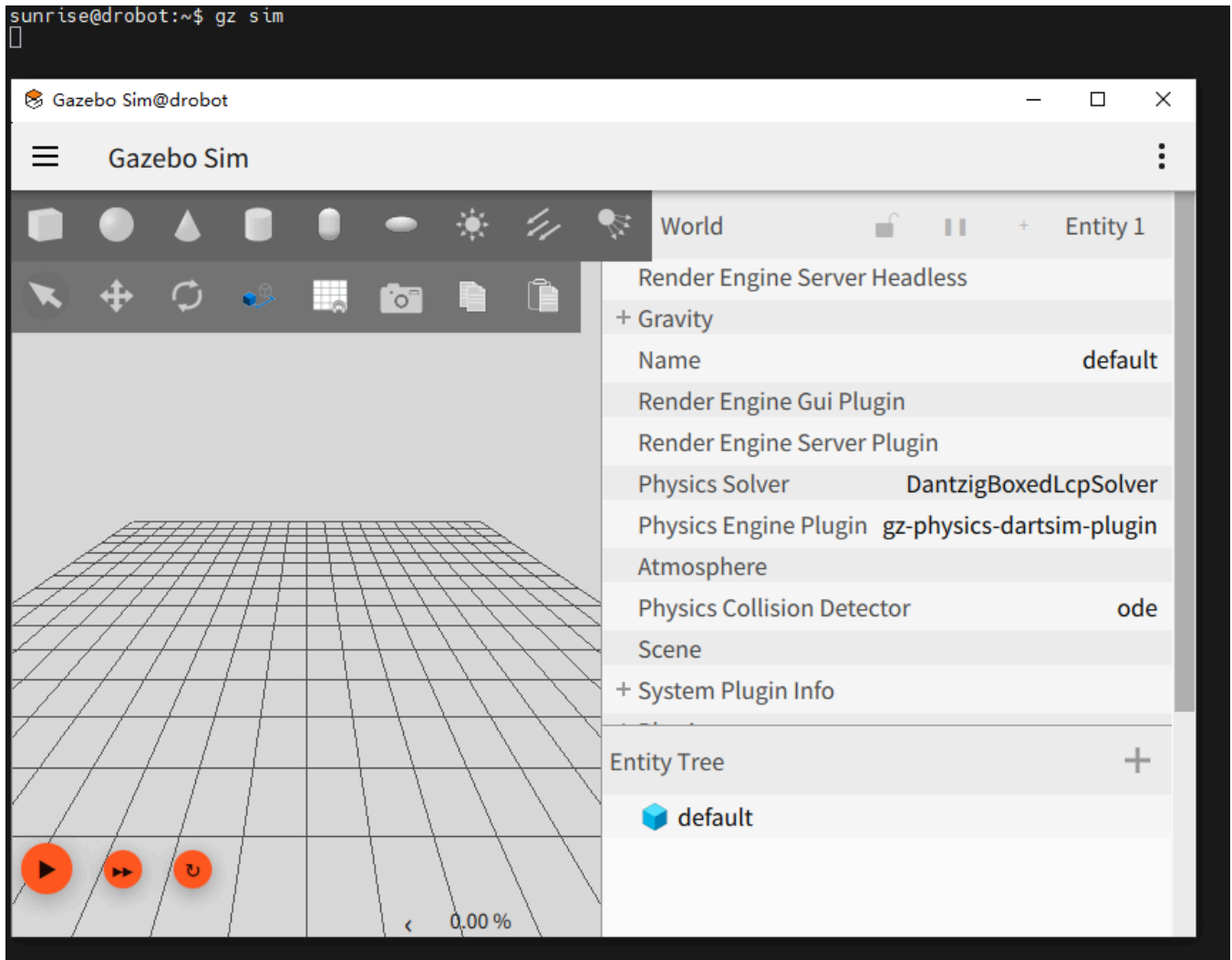
```
sudo apt install ros-jazzy-ros-gz
```

- Run gazebo:
- Start via the following command or directly through the desktop icon:

```
gz sim
```



After running, you can see the following page:



3. Create Package

- Create your own working directory. In the src directory of the workspace, create a myrobot package to store our SDF model files and launch files:

```
ros2 pkg create myrobot --build-type ament_cmake
```

- Go to the myrobot directory and create launch and sdf folders. Create a demo01_base.sdf file in the sdf folder. This file is a simple demonstration file with only a basic cube.

```
<sdf version='1.11'>
  <model name='myrobot'>
    <link name='base_link'>
      <inertial>
        <pose>0 0 0 0 0 0</pose>
        <mass>0.10000000000000001</mass>
        <inertia>
          <ixx>0.00019041666666700001</ixx>
          <ixy>0</ixy>
          <ixz>0</ixz>
          <iyy>0.00019039999999999999</iyy>
          <iyz>0</iyz>
          <izz>0.00036000000000000002</izz>
        </inertia>
      </link>
    </model>
  </sdf>
```

```

        </inertia>
    </inertial>
    <collision name='base_link_collision'>
        <pose>0 0 0 0 0 0</pose>
        <geometry>
            <box>
                <size>0.20000000000000001 0.20000000000000001 0.20000000000000001</size>
            </box>
        </geometry>
        <surface>
            <contact>
                <ode/>
            </contact>
            <friction>
                <ode/>
            </friction>
        </surface>
    </collision>
    <visual name='base_link_visual'>
        <pose>0 0 0 0 0 0</pose>
        <geometry>
            <box>
                <size>0.20000000000000001 0.20000000000000001 0.20000000000000001</size>
            </box>
        </geometry>
        <material>
            <script>
                <name>Gazebo/Red</name>
                <uri>file:///media/materials/scripts/gazebo.material</uri>
            </script>
        </material>
    </visual>
</link>
</model>
</sdf>

```

4. Write Launch File

For writing launch files, the launch file mainly starts two parts: starting the Gazebo file and loading the robot model into Gazebo.

```

start_gazebo = ExecuteProcess(
    cmd=['ros2', 'launch', 'ros_gz_sim', 'gz_sim.launch.py', 'gui:=true'],
    output='screen')

```

This command's purpose is to **start the new version Gazebo (Harmonic) simulation server and graphical interface**. It is equivalent to `gazebo --verbose -s ...` in classic Gazebo, but no longer requires manually loading ROS plugins because the new version's communication with ROS is entirely handled by independent "bridge nodes". The simulator itself is completely decoupled. `gui:=true` will open Gazebo's visualization

window at the same time, convenient for us to observe the simulation scene.

```
load_model = ExecuteProcess(  
    cmd=['ros2', 'run', 'ros_gz_sim', 'create',  
        '-file', sdf_model_path,  
        '-name', robot_name],  
    output='screen')
```

This line is equivalent to the `spawn_entity.py` node in classic Gazebo. Its function is to **generate a prepared robot model (in SDF format) into the simulation world**. The `-file` parameter specifies the model's SDF file path, and the `-name` parameter sets the unique name of the model in the simulation world.

Create a `bringup_model.launch.py` file in the launch directory. The complete launch file is as follows:

```
import os  
from launch import LaunchDescription  
from launch.actions import ExecuteProcess, TimerAction  
from launch_ros.substitutions import FindPackageShare  
  
def generate_launch_description():  
    robot_name = 'myrobot'  
    package_name = 'myrobot'  
    sdf_name = "demo01_base.sdf"    #SDF file  
  
    pkg_share = FindPackageShare(package=package_name).find(package_name)  
    sdf_model_path = os.path.join(pkg_share, f'sdf/{sdf_name}')  
  
    # 1. Start Gazebo simulation (with GUI)  
    start_gazebo = ExecuteProcess(  
        cmd=['ros2', 'launch', 'ros_gz_sim', 'gz_sim.launch.py', 'gui:=true'],  
        output='screen'  
    )  
  
    # 2. Load model into simulation (delay a few seconds to ensure Gazebo is ready)  
    load_model = ExecuteProcess(  
        cmd=['ros2', 'run', 'ros_gz_sim', 'create',  
            '-file', sdf_model_path,  
            '-name', robot_name],  
        output='screen'  
    )  
  
    delayed_load = TimerAction(  
        period=3.0,  
        actions=[load_model]  
    )  
  
    ld = LaunchDescription()  
    ld.add_action(start_gazebo)  
    ld.add_action(delayed_load)  
  
    return ld
```

- In CMakeLists.txt, add the following to install our sdf and launch folders into the install directory:

```
install(  
  DIRECTORY sdf launch  
  DESTINATION share/${PROJECT_NAME}  
)
```

- Then build and run the package:

```
colcon build --packages-select myrobot
```

```
sunrise@drobot:~/yhm_ws$ colcon build --packages-select myrobot  
Starting >>> myrobot  
Finished <<< myrobot [0.12s]  
  
Summary: 1 package finished [0.41s]  
sunrise@drobot:~/yhm_ws$
```

- Refresh environment variables and run the launch file:

```
source install/setup.bash
```

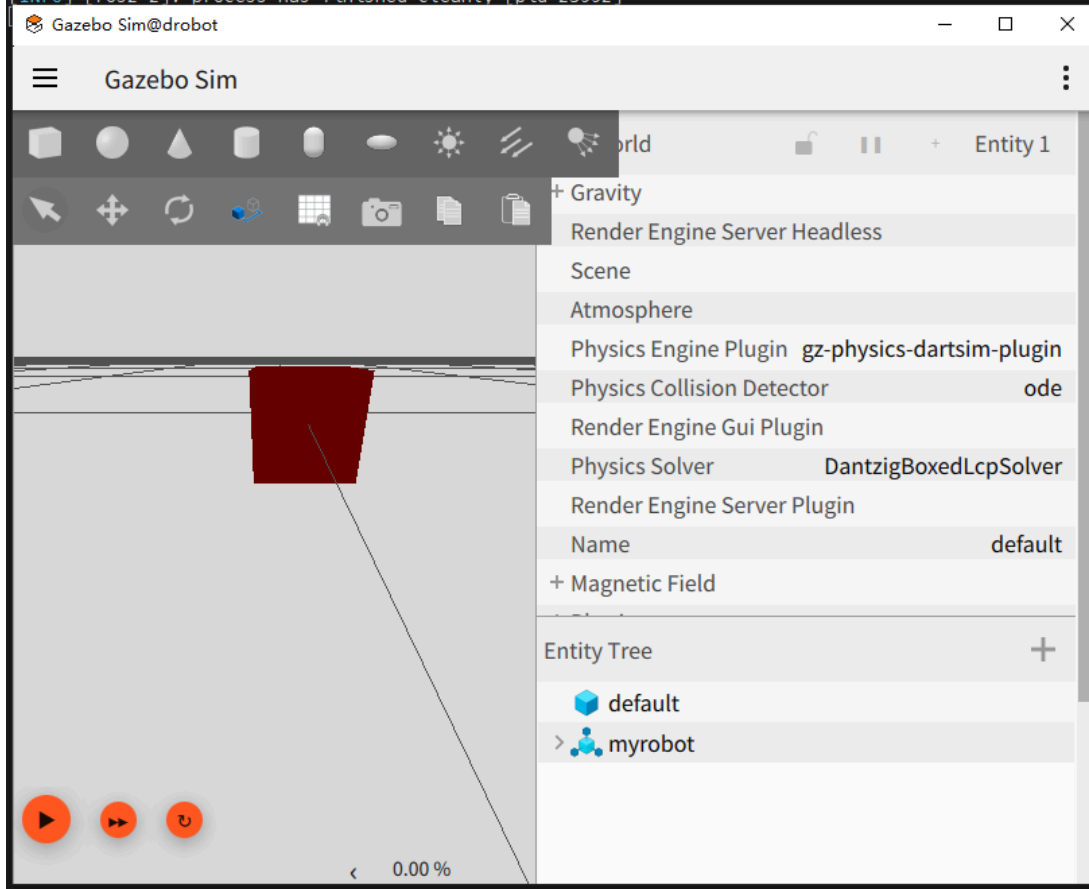
```
ros2 launch myrobot bringup_model.launch.py
```

After starting, you can see the following Gazebo model:

```

sunrise@drobot:~/yhm_ws$ source install/setup.bash
sunrise@drobot:~/yhm_ws$ ros2 launch myrobot bringup_model.launch.py
[INFO] [launch]: All log files can be found below /home/sunrise/.ros/log/2026-07-10-14-10-05-714558-drobot-23837
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [ros2-1]: process started with pid [23840]
[ros2-1] [INFO] [launch]: All log files can be found below /home/sunrise/.ros/log/2026-07-10-14-10-06-374944-drobot-23840
[ros2-1] [INFO] [launch]: Default logging verbosity is set to INFO
[ros2-1] [INFO] [gazebo-1]: process started with pid [23843]
[INFO] [ros2-2]: process started with pid [23992]
[ros2-2] [INFO] [1783663809.225543955] [ros_gz_sim]: Requesting list of world names.
[ros2-2] [INFO] [1783663809.453520207] [ros_gz_sim]: Entity creation successful.
[INFO] [ros2-2]: process has finished cleanly [pid 23992]

```



You can see the red model.